

SOAL DAN LEMBAR JAWAB

UAS PENGUJIAN PERANGKAT LUNAK

Nim : V3424002
Nama : ANANDITA PUTRI ARTIAR
Kelas : A
Email (quizizz/wayground) : ananditaputriartiar@student.uns.ac.id

Unit Testing

1. Skenario Pengujian : Sebuah program sederhana dibuat untuk menghitung diskon harga berdasarkan total pembelian.
 - a. Jika total pembelian kurang dari Rp 50.000, diskon 0%.
 - b. Jika total pembelian antara Rp 50.000 hingga Rp 100.000 (inklusif), diskon 5%.
 - c. Jika total pembelian lebih dari Rp 100.000, diskon 10%.

Rancanglah kasus uji (test cases) untuk fungsi ini menggunakan dua teknik *black-box* testing:

a. Equivalence Partitioning (Partisi Ekuivalen)

→ Equivalence Partitioning adalah teknik *black-box* testing yang membagi domain input ke dalam kelas-kelas yang dianggap “ekuivalen” artinya, jika satu nilai dalam kelas berhasil, semua nilai lain dalam kelas itu diasumsikan akan memberikan hasil yang sama.

Berdasarkan aturan diskon :

- **Kelas 1** : < Rp50.000 → diskon 0%
- **Kelas 2** : Rp50.000 - Rp100.000 (inklusif)→ diskon 5%
- **Kelas 3** : > Rp100.000 → diskon 10%

Test Cases :

Kelas	Contoh Input (total pembelian)	Expected Result (diskon)
1	Rp30.000	0%
2	Rp75.000	5%
3	Rp150.000	10%

Referensi: Myers, G. J., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing* (3rd ed.). Wiley. hlm. 43–47.

b. Boundary Value Analysis (Analisis Nilai Batas)

Sertakan input yang Anda uji dan hasil yang diharapkan (expected output).

→ Boundary Value Analysis (BVA) fokus pada nilai-nilai di batas transisi antar kelas, karena kesalahan sering terjadi di titik perubahan logika. Untuk setiap batas, kita uji nilai tepat di bawah, pada, dan di atas batas.

Batas:

- Batas 1 : Rp50.000
- Batas 2 : Rp100.000

Contoh Input (total pembelian)	Expected Output (diskon)
Rp49.999	0%
Rp50.000	5%
Rp50.001	5%
Rp99.999	5%
Rp100.000	5%
Rp100.001	10%

Referensi: Beizer, B. (1990). Software Testing Techniques (2nd ed.). International Thomson Computer Press. hlm. 25–32.

Integration Testing

2. Skenario Uji Integrasi Modul E-Commerce: Sebuah aplikasi e-commerce memiliki dua modul utama yaitu Modul Inventaris (Inventory Module) dan Modul Pemesanan (Ordering Module).

Modul Inventaris bertanggung jawab untuk melacak jumlah stok produk.

Modul Pemesanan bertanggung jawab untuk memproses pesanan pelanggan dan mengurangi stok yang tersedia saat pesanan berhasil dilakukan.

Tugas:

- a. Identifikasi Antarmuka/Integrasi: Jelaskan titik-titik integrasi utama antara kedua modul ini.

→ Titik integrasi utama meliputi:

- API calls: Modul Pemesanan memanggil fungsi **cekStok()** dan **kurangiStok()** dari Modul Inventaris.
- Shared database: Kedua modul membaca/tulis ke tabel **products.stock**.
- Event-driven communication: Order Service menerbitkan event seperti **OrderPlaced** yang dikonsumsi oleh Inventory Service.

- b. Buat Kasus Uji Positif: Rancang minimal 2 kasus uji integrasi untuk skenario "sukses" (misalnya, pemesanan produk yang tersedia).

→ Kasus Uji Positif (Sukses) :

Deskripsi	Data Uji	Hasil yang Diharapkan
Pesan produk dengan stok cukup	productId=101, stock=10, pesan=2	Stok berkurang menjadi 8

Pesan beberapa item dengan stok mencukupi	productId=102, stock=5, pesan=3	Stok berkurang menjadi 2
---	---------------------------------	--------------------------

- c. Buat Kasus Uji Negatif: Rancang minimal 2 kasus uji integrasi untuk skenario "gagal" (misalnya, mencoba memesan produk yang stoknya habis).

→ Kasus Uji Negatif (Gagal) :

Deskripsi	Data Uji	Hasil yang Diharapkan
Pesan produk dengan stok habis	productId=103, stock=0, pesan=1	Error : "Stok tidak tersedia"
Pesan melebihi stok	productId=104, stock=2, pesan=5	Error : "Stok tidak mencukupi"

- d. Tentukan Data Uji: Tentukan data uji spesifik yang diperlukan untuk menjalankan kasus uji negatif (misalnya, ProductID=101, CurrentStock=0).

→ Data Uji Spesifik untuk Kasus Negatif:

- **ProductID = 103, CurrentStock = 0**
- **ProductID = 104, CurrentStock = 2**

Referensi: Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill. hlm. 486–489.

3. Skenario Uji Integrasi Sistem Pembayaran Pihak Ketiga: Sebuah aplikasi pemesanan tiket bioskop mengintegrasikan layanan pembayaran eksternal (Payment Gateway). Ketika pengguna mengklik "Bayar Sekarang", aplikasi lokal berkomunikasi dengan API eksternal tersebut.

Tugas:

1. Jelaskan Strategi Testing, metode integrasi apa yang paling cocok digunakan di sini (Big Bang, Top-Down, atau Bottom-Up)? Jelaskan alasannya.

→ Strategi Testing yang Paling Cocok: Bottom-Up Testing

Alasan:

Karena Payment Gateway adalah layanan eksternal, integrasi dimulai dari komponen internal (modul aplikasi lokal) yang dipanggil terlebih dahulu. Layanan eksternal di-mock selama pengujian awal, sehingga Bottom-Up memungkinkan validasi logika tanpa ketergantungan pada sistem eksternal.

Referensi: ISTQB Foundation Level Syllabus v4.0 (2018). International Software Testing Qualifications Board. hlm. 42–44.

2. Rancang Kasus Uji API yang memvalidasi komunikasi dua arah:

- a. Mengirim permintaan pembayaran dengan detail kartu kredit yang valid dan memproses respons 200 OK (Sukses).
- b. Mengirim permintaan pembayaran dengan detail kartu kredit yang ditolak/kadaluarsa dan memproses kode kesalahan yang sesuai (4xx error).

Deskripsi	Payload/Request	Expected Respons
Kartu valid	{ "card": "4111111111111111", "exp": "12/27", "cvv": "123", "amount": 50000 }	HTTP 200, {"status": "success"}
Kartu ditolak	{ "card": "4000000000000002", "exp": "12/20", "cvv": "999", "amount": 50000 }	HTTP 402, {"error": "Card declined"}

Referensi: Stripe API Documentation – Test Cards. (2024).

<https://stripe.com/docs/testing>

3. Pertimbangkan Mocking / Stubbing, kapan Anda akan menggunakan stubs atau mocks dalam pengujian ini? Berikan satu contoh spesifik penggunaan mock untuk mengisolasi kegagalan.

→ Kapan digunakan?

Saat Payment Gateway tidak dapat diakses (offline, biaya transaksi nyata, atau rate limiting).

Contoh Penggunaan Mock:

Gunakan Mockito (Java) atau unittest.mock (Python) untuk mensimulasikan respons 402 Payment Required ketika kartu ditolak, sehingga pengujian tetap konsisten dan terisolasi.

Referensi: Fowler, M. (2007). *Mocks Aren't Stubs*.

<https://martinfowler.com/articles/mocksArentStubs.html>

4. Skenario Uji Aliran Data Microservis: Sebuah sistem pengiriman makanan terdiri dari tiga layanan mikro (microservices):
 - a. Layanan Pengguna (User Service): Mengelola profil pengguna dan alamat.
 - b. Layanan Pesanan (Order Service): Mengelola item pesanan dan statusnya.
 - c. Layanan Notifikasi (Notification Service): Mengirim email atau SMS konfirmasi.

Ketika pesanan berhasil dibuat (di Order Service), Order Service harus memicu Notification Service untuk mengirim konfirmasi ke alamat email yang diambil dari User Service.

Tugas:

1. Petakan Aliran Data: Gambarkan aliran data (atau jelaskan langkah-langkahnya) yang terjadi saat pesanan baru selesai.

→ Aliran Data saat Pesanan Dibuat

- User → Order Service: kirim permintaan pesanan.
- Order Service → User Service: *GET /users/{userId}* → dapatkan email.
- Order Service → Notification Service (via message queue/event): kirim data pesanan + email.
- Notification Service → kirim email konfirmasi.

Referensi: Newman, S. (2015). Building Microservices. O'Reilly. hlm. 87–92.

2. Rancang Rencana Uji Integrasi End-to-End (E2E): Buat rencana pengujian yang memvalidasi seluruh aliran, memastikan bahwa pembuatan pesanan di satu layanan menghasilkan email terkirim yang benar di layanan lain, menggunakan data pengguna yang tepat.

→ Rencana Uji Integrasi End-to-End (E2E)

- Langkah 1: Siapkan user dengan *email=user@example.com*.
- Langkah 2: Buat pesanan via API (*POST /orders*).
- Langkah 3: Gunakan email catcher (misal: MailHog) untuk verifikasi pengiriman email.
- Langkah 4: Validasi isi email (nomor pesanan, item, dll).

Referensi: Fowler, M. & Lewis, J. (2014). Microservices.

<https://martinfowler.com/articles/microservices.html>

3. Identifikasi Poin Kegagalan: Di mana potensi kegagalan integrasi terbesar dalam arsitektur ini (misalnya, koneksi jaringan, format data yang salah, latensi)?

→ Poin Kegagalan Potensial

- **Koneksi jaringan** antar layanan gagal (timeout).
- **Data format mismatch** (misal: User Service kirim *email* sebagai *null*).
- **Message queue down** → event tidak terkirim.
- **Autentikasi API gagal** (misal: JWT expired).

Referensi: Richardson, C. (2019). Microservices Patterns. Manning Publications. hlm. 112–118.

System Testing dan Acceptance Testing

5. Skenario uji : Anda sedang melakukan system testing untuk sebuah aplikasi mobile bank baru.

Pertanyaan:

a. Sebutkan dua (2) skenario fungsional utama yang perlu Anda uji.

→ Dua Skenario Fungsional Utama

- 1) Transfer antar rekening (internal & antar bank) → validasi saldo, notifikasi, dan log transaksi.

2) Cek saldo & riwayat transaksi → tampilan akurat dan real-time.

Referensi: ISO/IEC/IEEE 29119-1:2013 – Software and systems engineering — Software testing — Part 1: Concepts and definitions. hlm. 15.

b. Sebutkan satu (1) skenario non-fungsional (misal: kinerja atau keamanan) yang perlu diuji.

→ Satu Skenario Non-Fungsional

1) Keamanan: Autentikasi dua faktor (2FA), enkripsi data end-to-end, dan proteksi terhadap brute-force attack.

Referensi: OWASP Mobile Top 10 (2023).

<https://owasp.org/www-project-mobile-top-10/>

c. Sebutkan satu (1) jenis pengujian yang dilakukan setelah system testing (misalnya, acceptance testing) dan siapa pelakunya

→ Jenis Pengujian Setelah System Testing

- User Acceptance Testing (UAT)

Pelaku: Pengguna akhir (nasabah) atau perwakilan bisnis (tim produk/bank).

Referensi: IEEE Std 1012-2016 – Standard for System, Software, and Hardware Verification and Validation. hlm. 45.

6. Dalam menguji sebuah kelas "Manajer Database" yang memerlukan koneksi database aktif, bagaimana Anda akan menerapkan test fixture untuk skenario ini? Jelaskan langkah-langkah konkret, termasuk cara menyiapkan objek yang diperlukan sebelum pengujian dan membersihkan sumber daya setelah pengujian selesai.

→ Penerapan Test Fixture untuk Kelas "Manajer Database"

Langkah-langkah konkret:

1) Setup:

- Buat database in-memory (misal: H2 untuk Java, SQLite untuk Python).
- Jalankan skrip DDL/DML untuk membuat tabel dan seed data.
- Inisialisasi objek ManajerDatabase dengan koneksi ke DB uji.

2) Eksekusi: Jalankan test case (misal: insert, update, query).

3) Tear Down:

- Rollback transaksi atau hapus semua data.
- Tutup koneksi.
- Hancurkan DB in-memory.

Referensi: Meszaros, G. (2007). xUnit Test Patterns: Refactoring Test Code. Addison-Wesley. hlm. 217–225.